

Checkpoint 1 (data processing)

```
In [ ]: import warnings
warnings.filterwarnings("ignore", category=UserWarning)
import numpy as np
from tqdm import tqdm
import torch
import torch.nn as nn
import torchani
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

# Use GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device)

# Set up AEV computer
def init_aev_computer():
    Rcr = 5.2
    Rca = 3.5
    EtaR = torch.tensor([16], dtype=torch.float, device=device)
    ShfR = torch.tensor([
        0.900000, 1.168750, 1.437500, 1.706250,
        1.975000, 2.243750, 2.512500, 2.781250,
        3.050000, 3.318750, 3.587500, 3.856250,
        4.125000, 4.393750, 4.662500, 4.931250
    ], dtype=torch.float, device=device)

    EtaA = torch.tensor([8], dtype=torch.float, device=device)
    Zeta = torch.tensor([32], dtype=torch.float, device=device)
    ShfA = torch.tensor([0.90, 1.55, 2.20, 2.85], dtype=torch.float, device=device)
    ShfZ = torch.tensor([
        0.19634954, 0.58904862, 0.9817477, 1.37444680,
        1.76714590, 2.15984490, 2.5525440, 2.94524300
    ], dtype=torch.float, device=device)

    num_species = 4
    aev_computer = torchani.AEVComputer(
        Rcr, Rca, EtaR, ShfR, EtaA, Zeta, ShfA, ShfZ, num_species
    )
    return aev_computer

aev_computer = init_aev_computer()
aev_dim = aev_computer.aev_length
print(aev_dim)

# Prepare dataset & split
def load_ani_dataset(dspath):
    self_energies = torch.tensor([
        0.500607632585, -37.8302333826,
        -54.5680045287, -75.0362229210
```

```

], dtype=torch.float, device=device)
energy_shifter = torchani.utils.EnergyShifter(None)
species_order = ['H', 'C', 'N', 'O']

dataset = torchani.data.load(dspath)
dataset = dataset.subtract_self_energies(energy_shifter, species_order)
dataset = dataset.species_to_indices(species_order)
dataset = dataset.shuffle()
return dataset

```

```

In [2]: dataset = load_anl_dataset("ani_gdb_s01_to_s04.h5")
# Use dataset.split method to do split
train_data, val_data, test_data = dataset.split(0.8, 0.1, 0.1)

# Batching
batch_size = 8192
# use dataset.collate(...).cache() method to do batching
train_data_loader = train_data.collate(batch_size).cache()
val_data_loader = val_data.collate(batch_size).cache()
test_data_loader = test_data.collate(batch_size).cache()

#Torchani API
class AtomicNet(nn.Module):
    def __init__(self): #dropout_rate=0.1):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 128),
            nn.ReLU(),
            #nn.Dropout(dropout_rate),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.layers(x)

net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)

train_data_batch = next(iter(train_data_loader))

loss_func = nn.MSELoss()
species = train_data_batch['species'].to(device)
coords = train_data_batch['coordinates'].to(device)
true_energies = train_data_batch['energies'].to(device).float()
_, pred_energies = model((species, coords))

```

```
loss = loss_func(true_energies, pred_energies)
print(loss)
```

```
tensor(0.5897, device='cuda:0', grad_fn=<MseLossBackward0>)
```

Checkpoint 2 (model)

```
In [3]: class ANITrainer:
    def __init__(self, model, batch_size, learning_rate, epoch, 12):
        self.model = model

        num_params = sum(item.numel() for item in model.parameters())
        print(f"{model.__class__.__name__} - Number of parameters: {num_params}")

        self.batch_size = batch_size
        self.optimizer = torch.optim.Adam(model.parameters(), learning_rate, weight)
        self.epoch = epoch

    def train(self, train_data, val_data, early_stop=True, draw_curve=True):
        self.model.train()

        # init data loader
        print("Initialize training data...")
        train_data_loader = train_data

        # definition of Loss function: MSE is a good choice!
        loss_func = nn.MSELoss()

        # record epoch losses
        train_loss_list = []
        val_loss_list = []
        lowest_val_loss = np.inf

        for i in tqdm(range(self.epoch), leave=True):
            train_epoch_loss = 0.0
            for train_data_batch in train_data_loader:
                species = train_data_batch['species'].to(device)
                coordinates = train_data_batch['coordinates'].to(device)
                true_energies = train_data_batch['energies'].to(device).float()
                # compute energies
                _, pred_energies = self.model((species, coordinates))

                # compute loss
                batch_loss = loss_func(pred_energies, true_energies)

                # do a step
                self.optimizer.zero_grad()
                batch_loss.backward()
                self.optimizer.step()

                batch_importance = len(species)
                train_epoch_loss += batch_loss.item()*batch_importance

            # use the self.evaluate to get loss on the validation set
```

```

        val_epoch_loss = self.evaluate(val_data)

        # append the losses
        train_loss_list.append(train_epoch_loss)
        val_loss_list.append(val_epoch_loss)

        if early_stop:
            if val_epoch_loss < lowest_val_loss:
                lowest_val_loss = val_epoch_loss
                weights = self.model.state_dict()

    if draw_curve:
        fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
        ax.set_yscale("log")
        # Plot train loss and validation loss
        ax.plot(np.arange(self.epoch), train_loss_list, label='Train')
        ax.plot(np.arange(self.epoch), val_loss_list, label='Validation')
        ax.legend()
        ax.set_xlabel("# Batch")
        ax.set_ylabel("Loss")

    if early_stop:
        self.model.load_state_dict(weights)

    return train_loss_list, val_loss_list

def evaluate(self, data, draw_plot=False):

    # init data loader
    data_loader = data

    # init loss function
    loss_func = nn.MSELoss()
    total_loss = 0.0

    if draw_plot:
        true_energies_all = []
        pred_energies_all = []

    with torch.no_grad():
        for batch_data in data_loader:
            species = train_data_batch['species'].to(device)
            coordinates = train_data_batch['coordinates'].to(device)
            true_energies = train_data_batch['energies'].to(device).float()
            # compute energies
            _, pred_energies = self.model((species, coordinates))

            # compute loss
            batch_loss = loss_func(pred_energies, true_energies)

            batch_importance = len(species)
            total_loss += batch_loss.item() * batch_importance

        if draw_plot:
            true_energies_all.append(true_energies.detach().cpu().numpy().f

```

```

        pred_energies_all.append(pred_energies.detach().cpu().numpy().f

if draw_plot:
    true_energies_all = np.concatenate(true_energies_all)
    pred_energies_all = np.concatenate(pred_energies_all)
    # Report the mean absolute error
    # The unit of energies in the dataset is hartree
    # please convert it to kcal/mol when reporting the mean absolute error
    # 1 hartree = 627.5094738898777 kcal/mol
    # MAE = mean(|true - pred|)
    hartree2kcalmol = 627.5094738898777
    mae = np.mean(np.abs(true_energies_all - pred_energies_all))*hartree2kc
    fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
    ax.scatter(true_energies_all, pred_energies_all, label=f"MAE: {mae:.2f}")
    ax.set_xlabel("Ground Truth")
    ax.set_ylabel("Predicted")
    xmin, xmax = ax.get_xlim()
    ymin, ymax = ax.get_ylim()
    vmin, vmax = min(xmin, ymin), max(xmax, ymax)
    ax.set_xlim(vmin, vmax)
    ax.set_ylim(vmin, vmax)
    ax.plot([vmin, vmax], [vmin, vmax], color='red')
    ax.legend()

return total_loss

```

```

In [19]: trainer = ANITrainer(model, batch_size=128, learning_rate=2e-3, epoch=50, l2=1e-6)
         trainer.train(train_data_loader, val_data_loader)

```

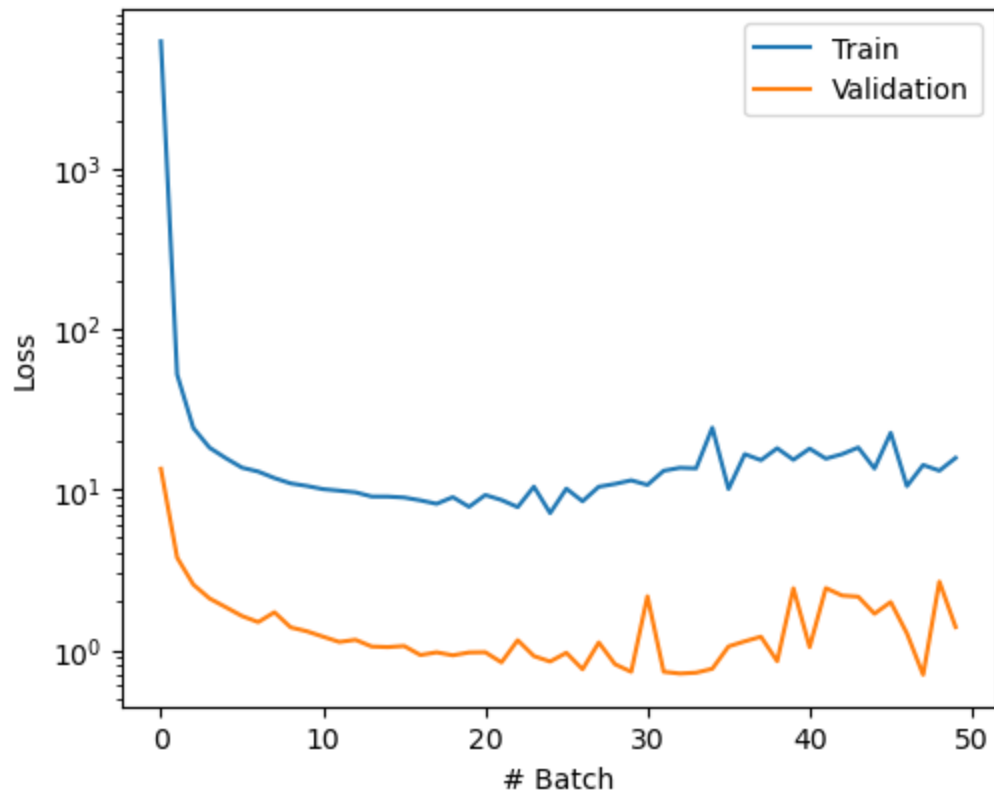
Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 50/50 [02:14<00:00, 2.68s/it]

```
Out[19]: ([6194.309649533097,  
52.10908511159505,  
24.191667689221504,  
18.23460150574101,  
15.704772057353694,  
13.697323806769418,  
12.953273436145537,  
11.797380558797158,  
10.930189311411596,  
10.537356234995968,  
10.065554259874261,  
9.849738074804918,  
9.614737057705497,  
9.054689585749657,  
9.04359604398087,  
8.926940123908935,  
8.569241478051481,  
8.155662624923934,  
8.977379046431452,  
7.811314966076679,  
9.254398450879307,  
8.616155078740121,  
7.78098834346747,  
10.43052461305706,  
7.142294697032412,  
10.137814787829484,  
8.458566661164696,  
10.420706310616879,  
10.847887511730733,  
11.419835700904514,  
10.699690448513138,  
13.100369917896387,  
13.670878516737503,  
13.561464045382309,  
24.304361934502595,  
10.089832638128428,  
16.62221188374133,  
15.24730204917978,  
18.096144730419837,  
15.339089746375976,  
17.998301691861343,  
15.587407154769608,  
16.627212795803644,  
18.28712482694391,  
13.532023220934207,  
22.5778989442897,  
10.51284105180821,  
14.236808225870845,  
13.114026898765587,  
15.750778279671067],  
[13.480128765106201,  
3.7706575989723206,  
2.5612446814775467,  
2.098521813750267,  
1.8521379828453064,  
1.6316495835781097,
```

1.4949180632829666,
1.7213267087936401,
1.386822834610939,
1.310638040304184,
1.2132397070527077,
1.1248397678136826,
1.1598066464066505,
1.0531006380915642,
1.0451436415314674,
1.061262309551239,
0.9325323402881622,
0.9691440984606743,
0.9287561178207397,
0.9672512710094452,
0.9705347791314125,
0.8392283767461777,
1.1536185592412949,
0.9184458702802658,
0.8493909537792206,
0.9628050401806831,
0.7597691938281059,
1.1161618009209633,
0.8172435089945793,
0.73284762352705,
2.1663181334733963,
0.73362947255373,
0.7151358649134636,
0.7239816263318062,
0.765819787979126,
1.05464768409729,
1.1347941309213638,
1.2123989164829254,
0.853361502289772,
2.428878605365753,
1.044372372329235,
2.4354991167783737,
2.188872277736664,
2.1523331105709076,
1.6858718991279602,
1.993762880563736,
1.2717368379235268,
0.7025652825832367,
2.670984536409378,
1.3940791189670563])



Checkpoint 3 (training, output analysis, visualization)

```
In [5]: # l2 regularization: changed l2 from 1e-6 to 1e-3
trainer = ANITrainer(model, batch_size=128, learning_rate=2e-3, epoch=50, l2=1e-3)
trainer.train(train_data_loader, val_data_loader)
```

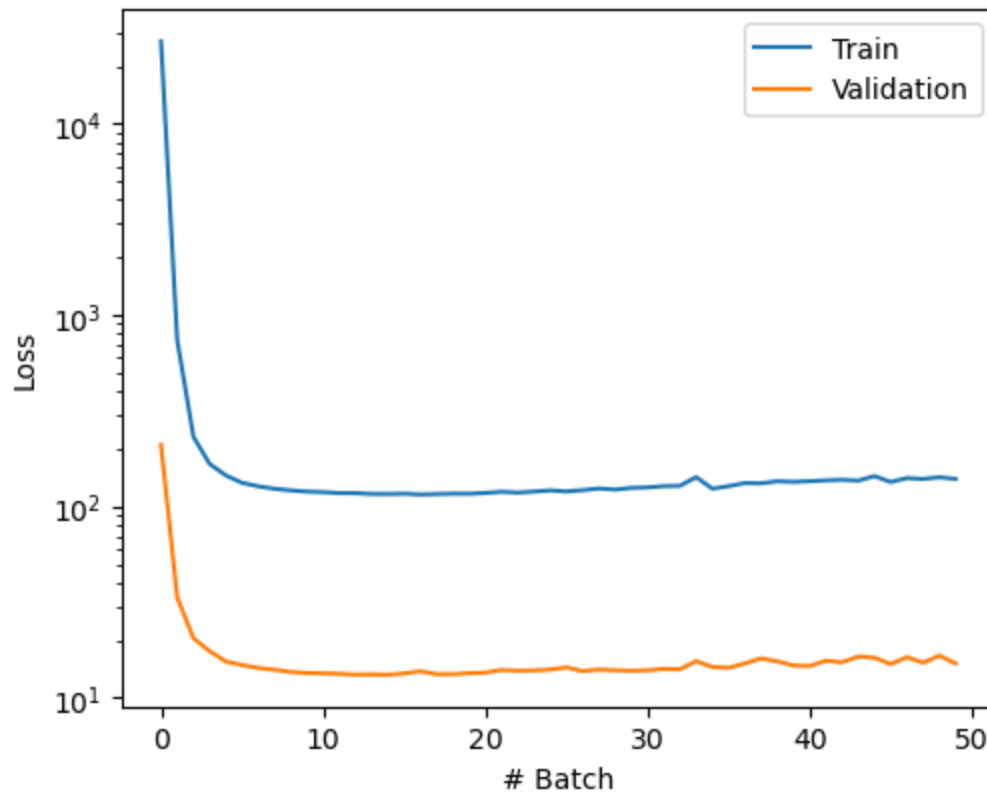
Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 50/50 [02:19<00:00, 2.78s/it]


```
Out[5]: ([26955.434332123492,  
738.9585825494723,  
231.18884066637838,  
166.69204997457564,  
145.22395495022647,  
132.74673546210397,  
127.4373733671382,  
123.65809329948388,  
121.29329312447226,  
119.49482955556596,  
118.803580479027,  
117.41923135443358,  
117.36048744962318,  
116.21446057292633,  
116.12509882255108,  
116.55779880005866,  
115.41845572099555,  
116.04154788921005,  
116.62477534689242,  
116.5183087727346,  
117.60295677158865,  
119.26579965412384,  
118.15434725474915,  
119.61037391345599,  
121.32731250292272,  
119.74956490873592,  
121.6578353450459,  
124.01698423401103,  
122.48464448066079,  
124.9896861181478,  
125.7087366489286,  
127.65549662074773,  
128.2058787761489,  
142.3222571759834,  
123.79524290980771,  
127.64214040886145,  
132.39374533150112,  
132.07493512652582,  
135.34813679609215,  
134.412331286876,  
135.4859926220961,  
136.90277595596854,  
137.77966006979113,  
136.49152270430932,  
144.04725174664054,  
134.15046050056117,  
140.61384260951309,  
139.22533778258367,  
142.0125074312673,  
139.3021657972131],  
[209.9193458557129,  
33.43350553512573,  
20.474903106689453,  
17.518629908561707,  
15.46064817905426,  
14.83877170085907,
```

14.310088634490967,
14.009374141693115,
13.644049406051636,
13.474446058273315,
13.418144464492798,
13.353281140327454,
13.215715169906616,
13.246195435523987,
13.192643761634827,
13.398704648017883,
13.766225218772888,
13.289152264595032,
13.275618076324463,
13.44574785232544,
13.532838702201843,
13.983239531517029,
13.85255777835846,
13.902779698371887,
14.041796207427979,
14.415381073951721,
13.78937315940857,
14.03620457649231,
13.93412470817566,
13.845651984214783,
13.92457103729248,
14.156357049942017,
14.080907106399536,
15.548234224319458,
14.483754873275757,
14.353599429130554,
15.105459332466125,
16.06065082550049,
15.509158611297607,
14.743791222572327,
14.664801716804504,
15.619998216629028,
15.334301948547363,
16.430991411209106,
16.198820114135742,
14.999549627304077,
16.299562215805054,
15.24268352985382,
16.601356625556946,
15.129555225372314])



The loss curve with original architecture has a low validation loss but it increases and wavers in later batches. The lowest point of the loss curve is around $10^0=1$. Since an increasing validation loss in later batches is likely a sign of overfitting, I increased the l2 regularization value from $1e-6$ to $1e-3$, which is expected to reduce overfitting. However, the validation loss increased to around 10^1 , so I kept l2 as $1e-6$ in later trials.

```
In [9]: # Dropout: added the line "nn.Dropout(dropout_rate)" in the AtomicNet class
trainer = ANITrainer(model, batch_size=128, learning_rate=2e-3, epoch=50, l2=1e-6)
trainer.train(train_data_loader, val_data_loader)
```

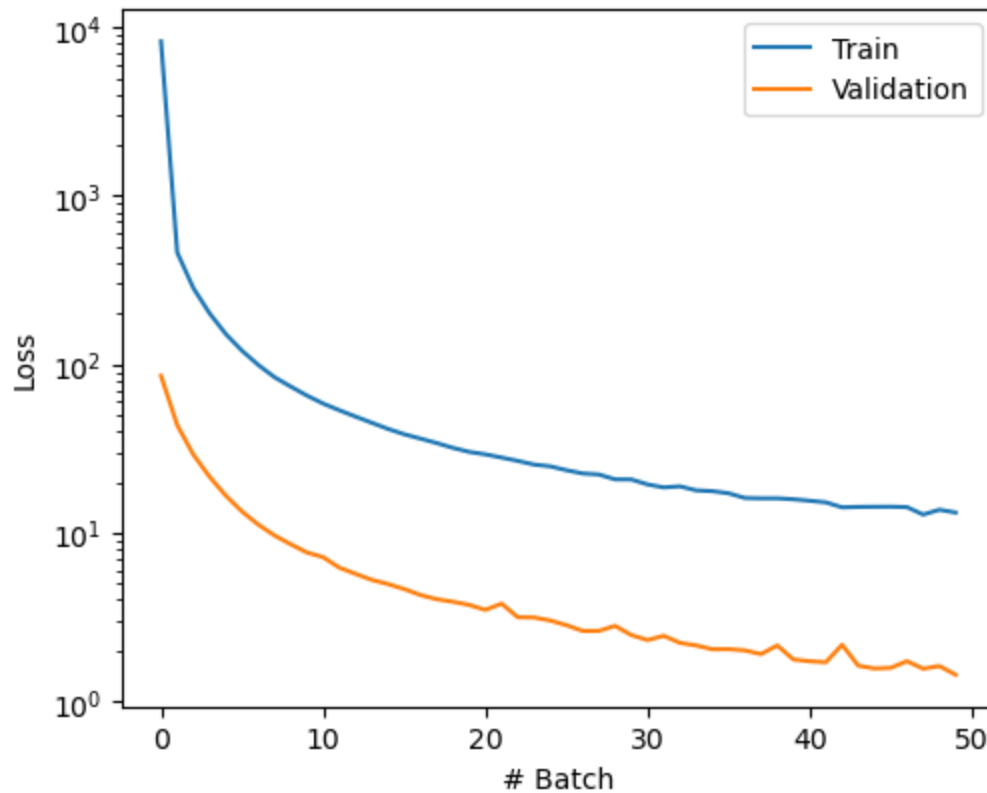
Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 50/50 [02:16<00:00, 2.73s/it]

```
Out[9]: ([8253.48469635658,  
         461.256382547901,  
         282.8684203872108,  
         200.530182638322,  
         151.1953908444557,  
         120.39021285067429,  
         99.18286274932325,  
         83.91646062319342,  
         73.804865934173,  
         65.27688045258401,  
         58.52280540045467,  
         53.40105367748765,  
         48.98005783073313,  
         45.01276101909025,  
         41.39941843894485,  
         38.481534687642124,  
         36.26312726930337,  
         34.1300686015893,  
         31.964192429477407,  
         30.24939900924801,  
         29.25033858855022,  
         27.98668413964333,  
         26.75128379650414,  
         25.439590631016472,  
         24.833625973893504,  
         23.551943409242085,  
         22.52772231245035,  
         22.209469167661155,  
         20.78519615663754,  
         20.75584518267715,  
         19.39466064541557,  
         18.61899040703429,  
         18.890990213916666,  
         17.876153138207883,  
         17.69162220513681,  
         17.181793617903168,  
         16.11491265261793,  
         16.03559828354264,  
         16.041005328756,  
         15.833575677836052,  
         15.51774229336661,  
         15.176037826706306,  
         14.183444328842597,  
         14.265015356977528,  
         14.288785865952377,  
         14.300024519940052,  
         14.2272526005072,  
         12.844777194648486,  
         13.723584557137656,  
         13.204886346215062],  
         [85.55376386642456,  
         43.5560028553009,  
         29.088955879211426,  
         21.50183618068695,  
         16.619741201400757,  
         13.376093626022339,
```

11.198552966117859,
9.651302695274353,
8.561156868934631,
7.651269257068634,
7.172854959964752,
6.217185318470001,
5.711589455604553,
5.262709259986877,
4.972051709890366,
4.640607714653015,
4.279384672641754,
4.042668908834457,
3.903846949338913,
3.7384398579597473,
3.4908436834812164,
3.8066375851631165,
3.1610173881053925,
3.1492032408714294,
3.022755056619644,
2.8340159505605698,
2.6216255575418472,
2.6218458265066147,
2.808834508061409,
2.4777273535728455,
2.3124859631061554,
2.4519076496362686,
2.2259209901094437,
2.1492511183023453,
2.04005129635334,
2.0425955057144165,
2.008839175105095,
1.9088476300239563,
2.1499763429164886,
1.7748105973005295,
1.731770470738411,
1.7086893767118454,
2.169780597090721,
1.629271611571312,
1.5689908266067505,
1.5822950452566147,
1.7332552671432495,
1.5641788095235825,
1.6228349208831787,
1.4398072585463524])



I tried adding dropout (added the line "nn.Dropout(dropout_rate)" in the AtomicNet class, and set dropout rate = 0.1), which is also used to prevent overfitting by setting a number of nodes to 0. The resulting validation loss is higher than without dropout, at around 1.1, so I removed the dropout line from AtomicNet in later trials.

```
In [11]: # Hyperparameter tuning: changed Learning rate from 2e-3 to 1e-4
trainer = ANITrainer(model, batch_size=128, learning_rate=1e-4, epoch=50, l2=1e-6)
trainer.train(train_data_loader, val_data_loader)
```

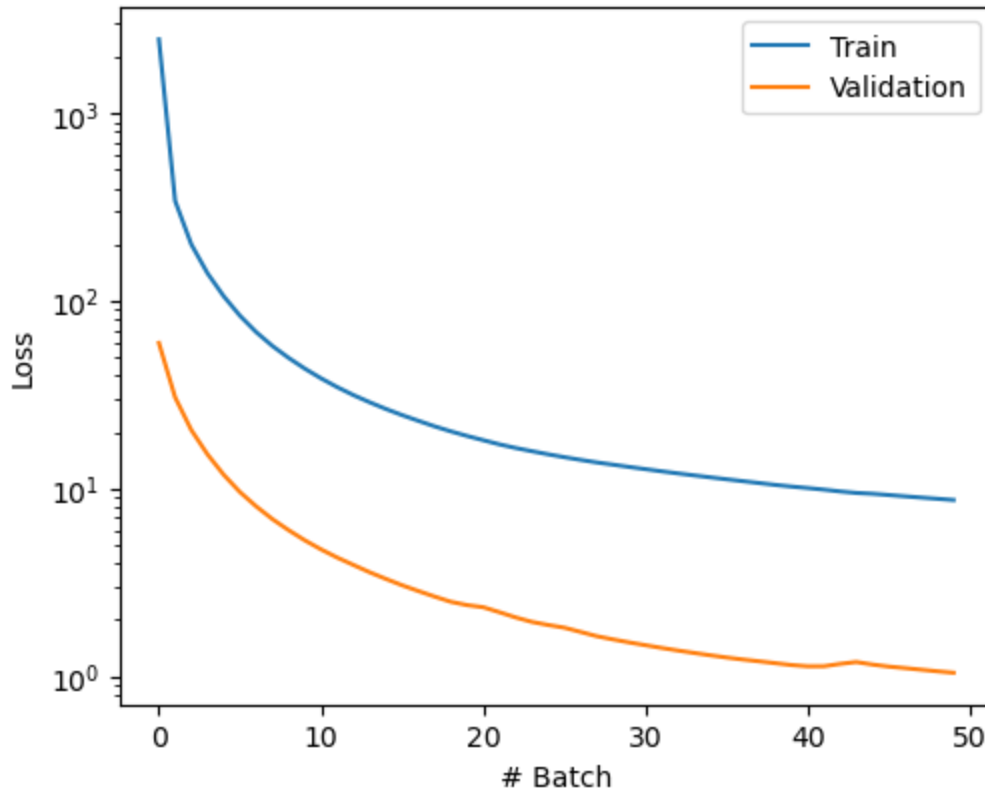
Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 50/50 [02:14<00:00, 2.70s/it]

```
Out[11]: ([2484.3966208309866,  
341.8580849347636,  
200.2816405465128,  
140.4543411647901,  
105.90937490749639,  
83.43455492795329,  
68.23328316984407,  
57.571921225622646,  
49.73579847243673,  
43.57373435301997,  
38.68677833140828,  
34.721700066867925,  
31.488302324258257,  
28.834114368568407,  
26.55845949127979,  
24.633855353065883,  
22.969692364735238,  
21.46548348081342,  
20.18082434932512,  
19.078433257520373,  
18.107948512846633,  
17.202080204646336,  
16.456586074749794,  
15.81813616902582,  
15.227989671824616,  
14.700878141276917,  
14.2209706456415,  
13.7822025672358,  
13.409247434421559,  
13.038221837294259,  
12.691933249810972,  
12.3577080267969,  
12.047417764551938,  
11.75660933344625,  
11.476578472391338,  
11.209251819265774,  
10.953263347457323,  
10.704045124808545,  
10.46069873930719,  
10.25010330094483,  
10.065239497140283,  
9.869741977056037,  
9.655923425916626,  
9.477742717619549,  
9.38241861363531,  
9.238709757148172,  
9.090571760769308,  
8.95378056010668,  
8.82079734526269,  
8.689596347599945],  
[59.7565016746521,  
30.648743391036987,  
20.499607920646667,  
15.198934197425842,  
11.83500051498413,  
9.574509501457214,
```

8.034635603427887,
6.872602224349976,
5.999270439147949,
5.295630186796188,
4.738439559936523,
4.284120410680771,
3.9129609763622284,
3.576779007911682,
3.296954929828644,
3.0500932931900024,
2.8417552411556244,
2.6554425954818726,
2.490644246339798,
2.3911522775888443,
2.3299641758203506,
2.1910726875066757,
2.055279642343521,
1.9456377029418945,
1.8732722997665405,
1.8122185319662094,
1.7203629165887833,
1.6323263198137283,
1.5693768709897995,
1.5123120099306107,
1.461619794368744,
1.4134999215602875,
1.3693306371569633,
1.3289208114147186,
1.2912518680095673,
1.2567616999149323,
1.2261580899357796,
1.1996252536773682,
1.1724123805761337,
1.1457411721348763,
1.128208115696907,
1.1285167261958122,
1.1620347127318382,
1.1889849677681923,
1.1525352597236633,
1.1242868527770042,
1.103573516011238,
1.0819916799664497,
1.0609399676322937,
1.0420201420783997])



The learning rate hyperparameter determines how much the model changes in response to the error. I decreased the learning rate from $2e-3$ to $1e-4$, which should allow the model to change more gradually, may cause it to converge slowly, but can prevent overfitting. The resulting loss curve did not oscillate anymore in later batches, so I kept the learning rate at $1e-4$ in subsequent trials.

```
In [12]: # Hyperparameter tuning: changed batch size from 128 to 256
trainer = ANITrainer(model, batch_size=256, learning_rate=1e-4, epoch=50, l2=1e-6)
trainer.train(train_data_loader, val_data_loader)
```

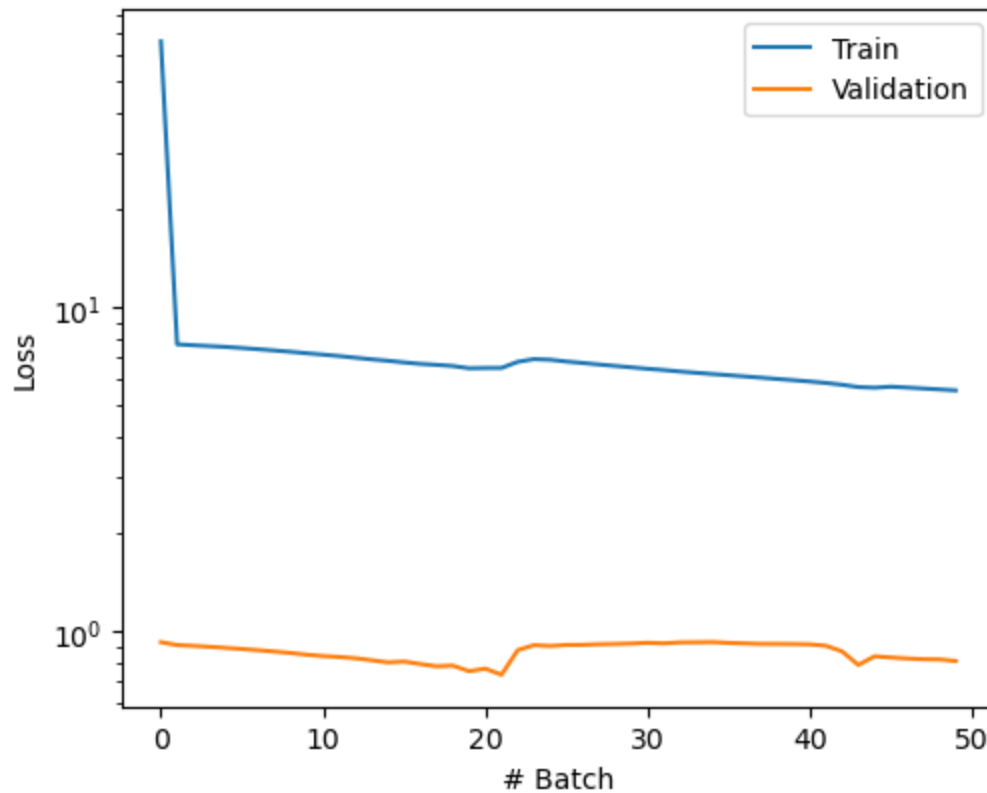
Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 50/50 [02:15<00:00, 2.71s/it]

```
Out[12]: ([66.13955136010554,  
7.690453991646791,  
7.637412332778695,  
7.593396834829036,  
7.548071742992761,  
7.488339594336139,  
7.42643509791742,  
7.3601541521493345,  
7.291482439961328,  
7.216470479925192,  
7.145260075241822,  
7.073379389450565,  
6.990031210967572,  
6.910444165851004,  
6.840947723439967,  
6.760038316380815,  
6.69057684452855,  
6.644991852726889,  
6.590925774360585,  
6.485454577054043,  
6.4981908571371605,  
6.500862006794705,  
6.789396582451445,  
6.921516940879883,  
6.891345435897165,  
6.804152751730726,  
6.735256109532202,  
6.661813322689341,  
6.59550933819628,  
6.52880466449642,  
6.462428690891102,  
6.405387407296075,  
6.338316774921623,  
6.281279102182452,  
6.225915047174567,  
6.175204708906676,  
6.120506935576486,  
6.066837685506471,  
6.012032823058689,  
5.9593749758550985,  
5.904338645259486,  
5.844088846342856,  
5.768936657259474,  
5.675381053704768,  
5.649970468719403,  
5.689730859518022,  
5.656418012820723,  
5.618157090109889,  
5.581200011965848,  
5.543876904503122],  
[0.9268540367484093,  
0.908031702041626,  
0.9029871076345444,  
0.8970372453331947,  
0.8901310190558434,  
0.8832139298319817,
```

0.8756903856992722,
0.8672331869602203,
0.8582272157073021,
0.8478101938962936,
0.8397879898548126,
0.8344617635011673,
0.8269567862153053,
0.8151522874832153,
0.8030196949839592,
0.8077314719557762,
0.7934146672487259,
0.7810171246528625,
0.7845589444041252,
0.7543055415153503,
0.7680210098624229,
0.7353965118527412,
0.8758731558918953,
0.9073802828788757,
0.9021498337388039,
0.908745288848877,
0.9094120487570763,
0.9131610170006752,
0.9151642620563507,
0.9180235639214516,
0.9229244142770767,
0.9192957058548927,
0.9253252893686295,
0.9260292202234268,
0.9273987337946892,
0.9220948442816734,
0.9189035668969154,
0.9155010804533958,
0.9156520813703537,
0.9144997224211693,
0.9128715246915817,
0.9036756232380867,
0.8680505007505417,
0.7885259389877319,
0.8383314162492752,
0.83171696215868,
0.8256246075034142,
0.8211069032549858,
0.8201607167720795,
0.8109147399663925])



I tried using a larger batch size (256 instead of 128), to allow the model to train more quickly and progress to a better accuracy. This led to a loss curve that stayed below 10^0 , which means lower losses than a smaller batch size, so I will keep the batch size at 256.

My final choice model has `batch_size=256`, `learning_rate=1e-4`, `epoch=50`, and `l2=1e-6`. I kept the original small l2 value because it resulted in lower validation loss compared to increasing l2. I tried using dropout but it increased the loss, so I ended up excluding dropout. Decreasing the learning rate resulted in less oscillation in later batches, and increasing the batch size kept the loss consistently low, so I kept these changes to the two hyperparameters.

Checkpoint 4 (training, output analysis, visualization)

In [24]: *# adapted from HW 6*

```
from sklearn.model_selection import KFold

def ANI_KFold(model, k, dataset, batch_size=32, learning_rate=1e-4, epoch=50, l2=1e-6):
    kf = KFold(n_splits=k, shuffle=True)
    list_dataset = list(dataset)
    subset_indices = range(1000)
    subset_dataset = [list_dataset[m] for m in subset_indices]
    train_loss_list, val_loss_list = [], []
    for i, (train_idx, val_idx) in enumerate(kf.split(subset_dataset)):
```

```

print(f"Fold {i}:")

train_dataset = [subset_dataset[j] for j in train_idx]
train_dataset = torchani.data.TransformableIterable(train_dataset)
val_dataset = [subset_dataset[j] for j in val_idx]
val_dataset = torchani.data.TransformableIterable(val_dataset)

train_data_loader = train_dataset.collate(batch_size)
val_data_loader = val_dataset.collate(batch_size)

net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)

trainer = ANITrainer(model, batch_size, learning_rate, epoch, 12)
train_losses, val_losses = trainer.train(train_data_loader, val_data_loader)
train_loss_list.append(np.min(train_losses))
val_loss_list.append(np.min(val_losses))

print("Final results:")
print(f"Train loss: {np.mean(train_loss_list)}+/-{np.std(train_loss_list)}")
print(f"Validation loss: {np.mean(val_loss_list)}+/-{np.std(val_loss_list)}")

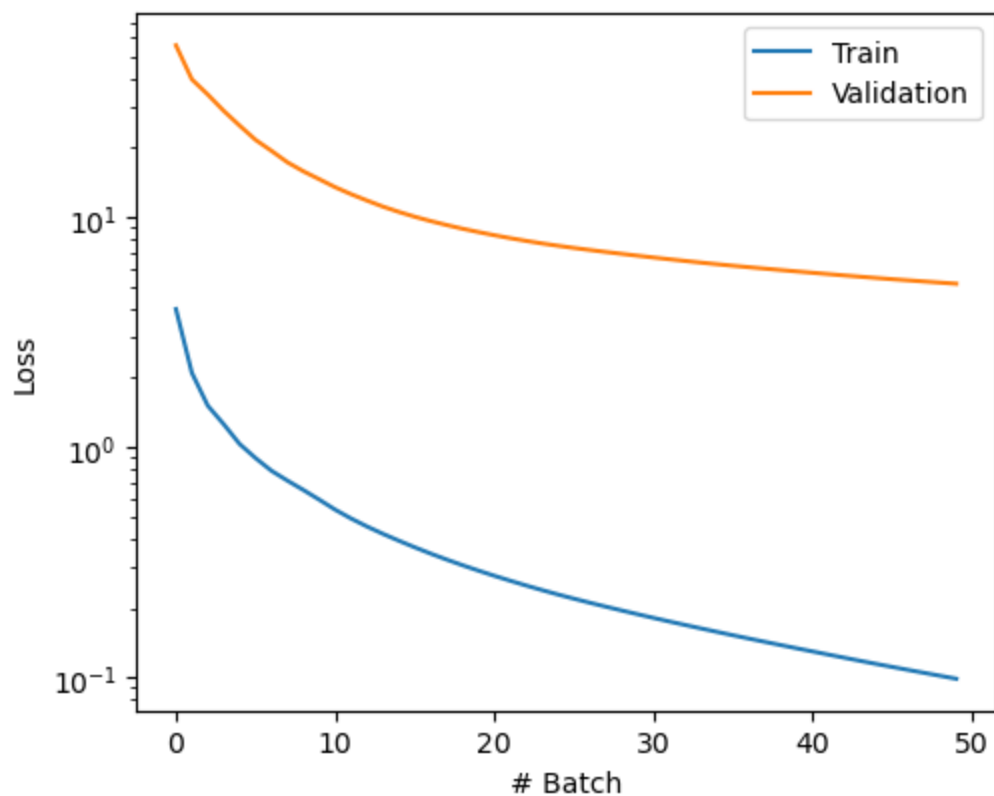
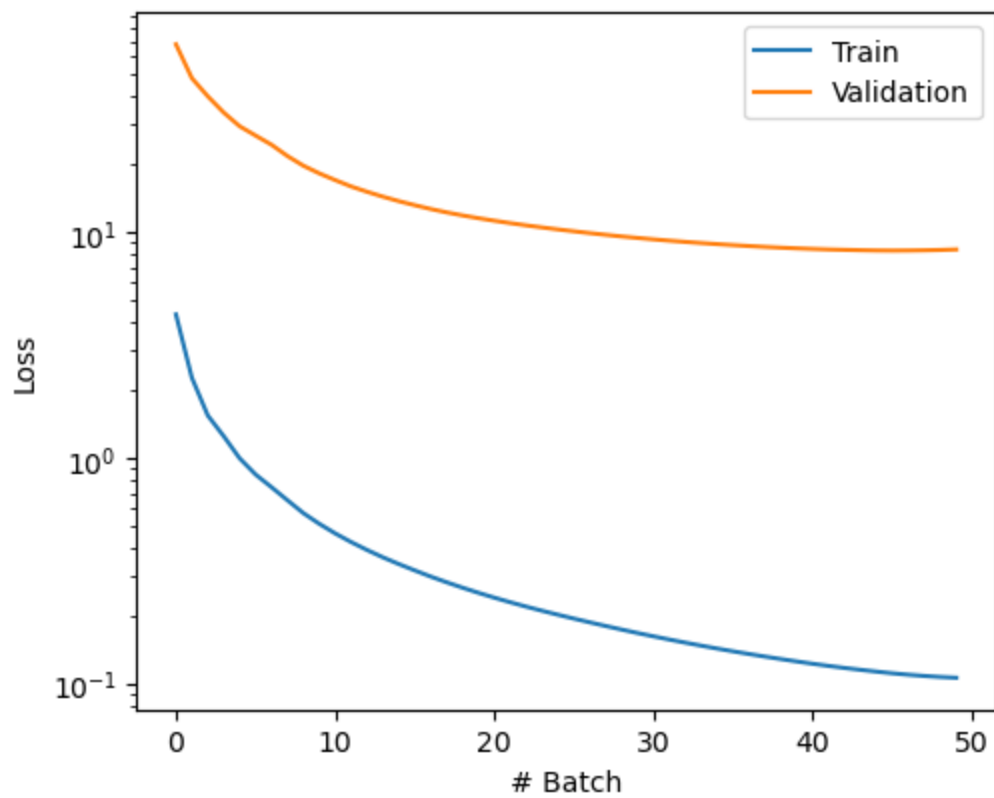
```

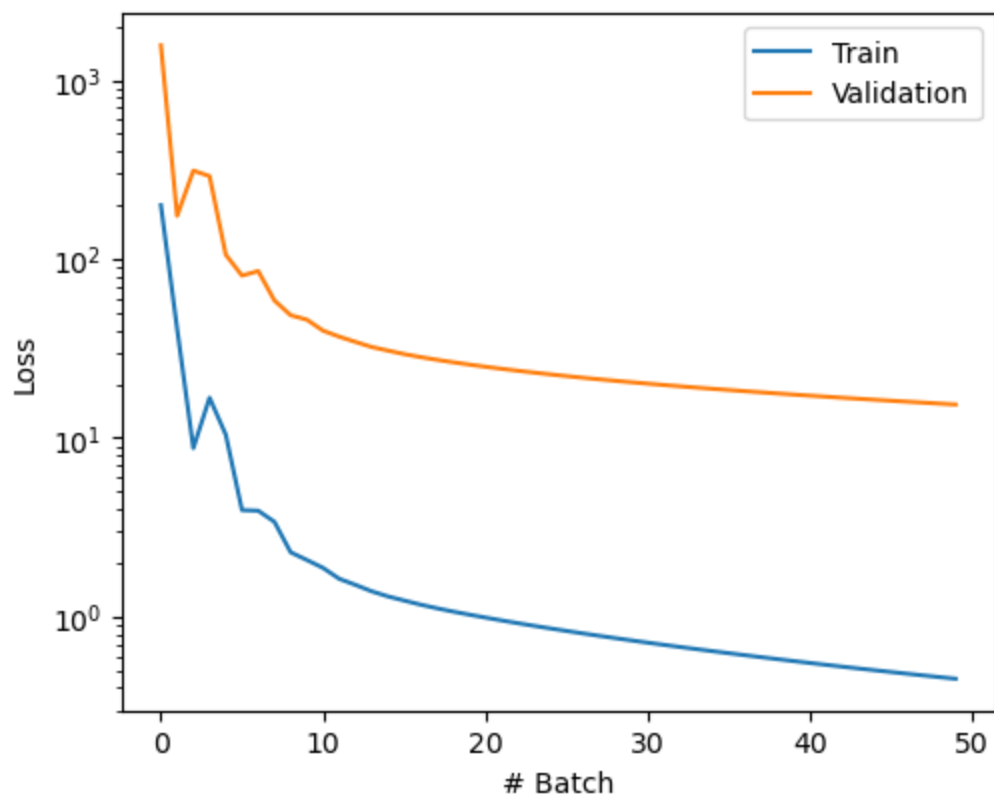
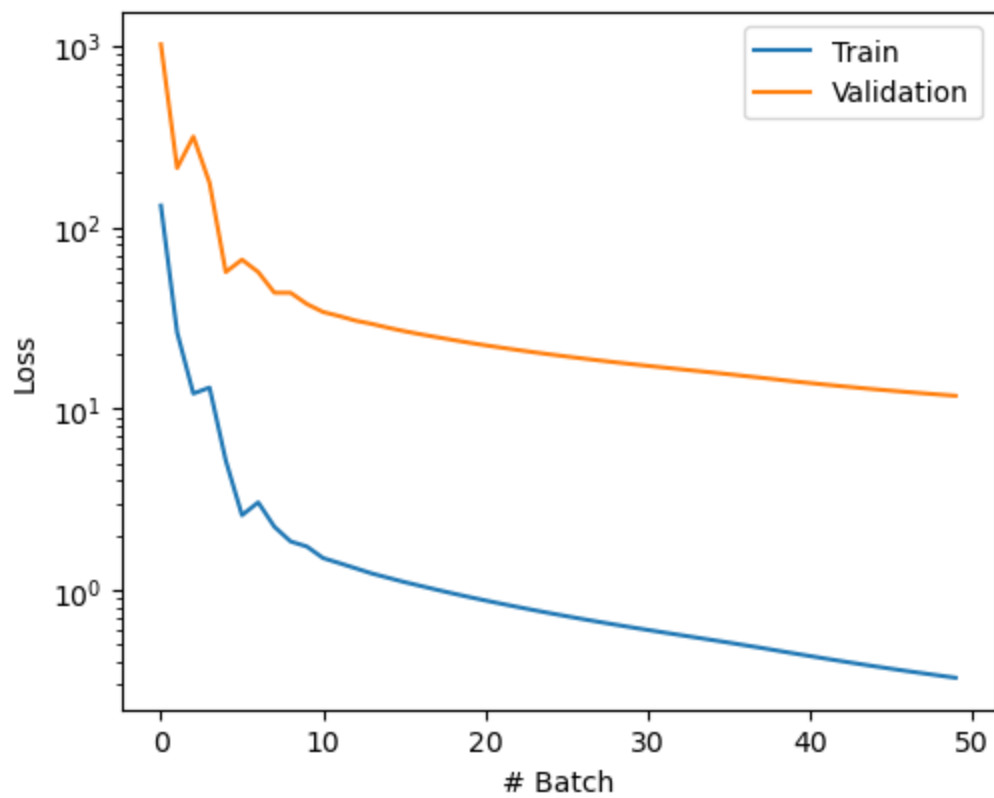
In [25]: ANI_KFold(model=AtomicNet, k=5, dataset=dataset, batch_size=128, learning_rate=1e-4

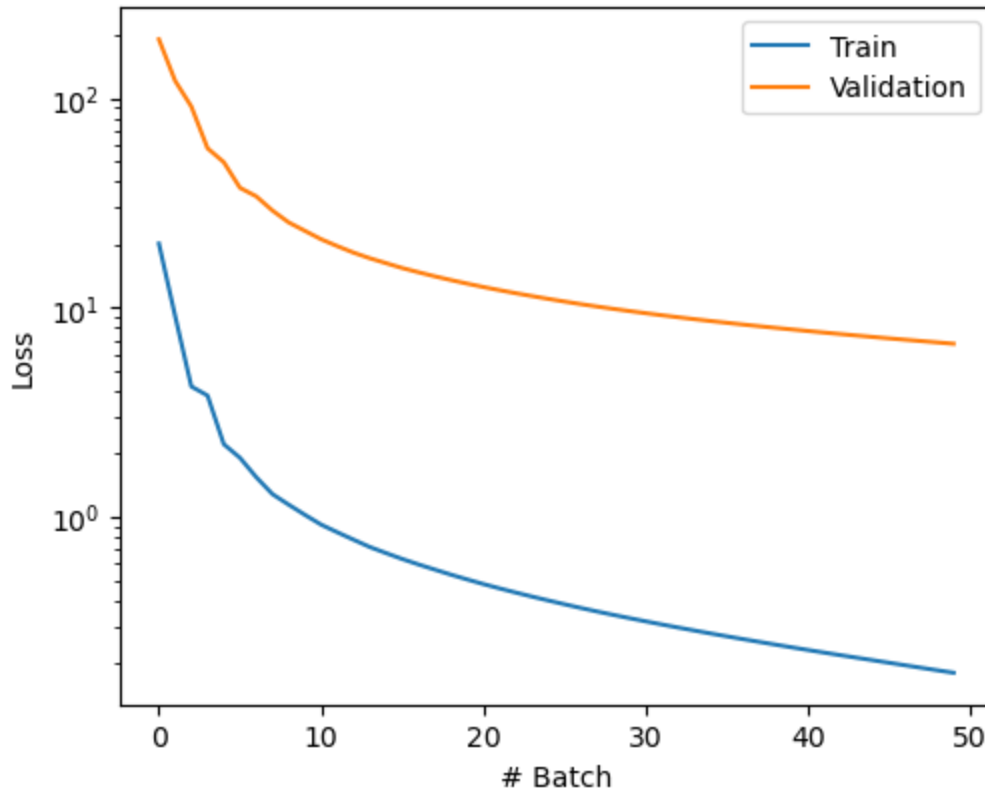
```

Fold 0:
Sequential - Number of parameters: 197636
Initialize training data...
100%|██████████| 50/50 [00:05<00:00, 9.25it/s]
Fold 1:
Sequential - Number of parameters: 197636
Initialize training data...
100%|██████████| 50/50 [00:05<00:00, 8.70it/s]
Fold 2:
Sequential - Number of parameters: 197636
Initialize training data...
100%|██████████| 50/50 [00:05<00:00, 9.36it/s]
Fold 3:
Sequential - Number of parameters: 197636
Initialize training data...
100%|██████████| 50/50 [00:05<00:00, 9.21it/s]
Fold 4:
Sequential - Number of parameters: 197636
Initialize training data...
100%|██████████| 50/50 [00:05<00:00, 9.34it/s]
Final results:
Train loss: 0.23288204159471207+/-0.13646660679559516
Validation loss: 9.458785152435302+/-3.681712099076994

```







Used the settings chosen in checkpoint 3, except with the previous smaller batch size of 128 rather than 256 because the loss curve for 256 does not look normal. I also trained on a subset of the data because the whole set took too long to compute.

Checkpoint 5 (training, output analysis, visualization)

In [16]: *# adapted from HW 6*

```
from sklearn.model_selection import KFold
from sklearn.model_selection import train_test_split

def ANI_KFold(model, k, dataset, batch_size=32, learning_rate=1e-4, epoch=5, l2=1e-4):
    kf = KFold(n_splits=k, shuffle=True)
    list_dataset = list(dataset)
    trainval_dataset, test_dataset = train_test_split(list_dataset, test_size=0.15)
    train_loss_list, val_loss_list = [], []

    for i, (train_idx, val_idx) in enumerate(kf.split(trainval_dataset)):
        print(f"Fold {i}:")

        train_dataset = [trainval_dataset[j] for j in train_idx]
        train_dataset = torchani.data.TransformableIterable(train_dataset)
        val_dataset = [trainval_dataset[j] for j in val_idx]
        val_dataset = torchani.data.TransformableIterable(val_dataset)

        train_data_loader = train_dataset.collate(batch_size)
```



```

val_data_loader = val_dataset.collate(batch_size)

net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)

trainer = ANITrainer(model, batch_size, learning_rate, epoch, 12)
train_losses, val_losses = trainer.train(train_data_loader, val_data_loader)
train_loss_list.append(np.min(train_losses))
val_loss_list.append(np.min(val_losses))

test_dataset = torchani.data.TransformableIterable(test_dataset)
test_data_loader = test_dataset.collate(batch_size)
test_loss = trainer.evaluate(test_data_loader)

print("Final results:")
print(f"Train loss: {np.mean(train_loss_list)}+/-{np.std(train_loss_list)}")
print(f"Train RMSE: {np.sqrt(np.mean(train_loss_list))}+/-{np.sqrt(np.std(train_loss_list))}")
print(f"Validation loss: {np.mean(val_loss_list)}+/-{np.std(val_loss_list)}")
print(f"Validation RMSE: {np.sqrt(np.mean(val_loss_list))}+/-{np.sqrt(np.std(val_loss_list))}")
print(f"Test loss: {test_loss}")
print(f"Test RMSE: {np.sqrt(test_loss)}")

```

```

In [ ]: # tried with run time too long
ANI_KFold(model=AtomicNet, k=5, dataset=dataset, batch_size=128, learning_rate=1e-4

```

Fold 0:

Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 50/50 [1:12:14<00:00, 86.70s/it]

Fold 1:

Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 50/50 [1:11:32<00:00, 85.86s/it]

Fold 2:

Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 50/50 [1:11:14<00:00, 85.50s/it]

Fold 3:

Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 50/50 [1:10:38<00:00, 84.76s/it]

Fold 4:

Sequential - Number of parameters: 197636

Initialize training data...

72%|██████████| 36/50 [50:34<20:06, 86.19s/it]

```
In [17]: # tried again with larger batch size and less epochs
ANI_KFold(model=AtomicNet, k=2, dataset=dataset, batch_size=512, learning_rate=1e-4
```

Fold 0:

Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 10/10 [06:10<00:00, 37.05s/it]

Fold 1:

Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 10/10 [06:11<00:00, 37.12s/it]

Final results:

Train loss: 5.770655643009832+/-0.025754561032499623

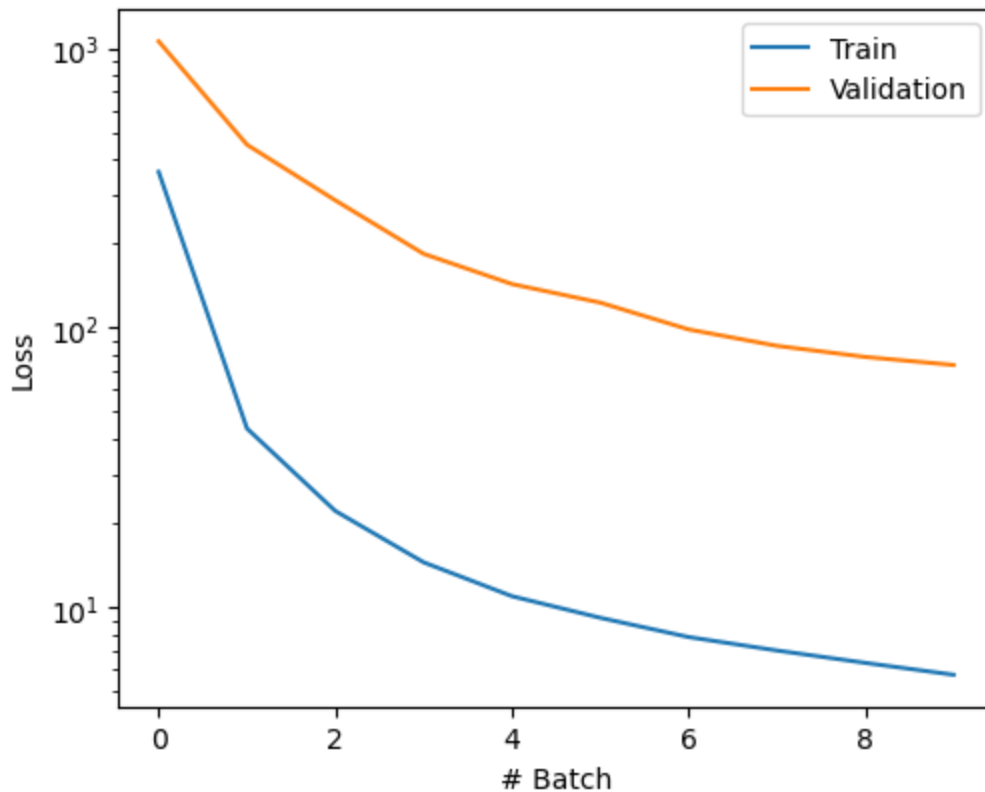
Train RMSE: 2.4022188998943936+/-0.16048227638122417

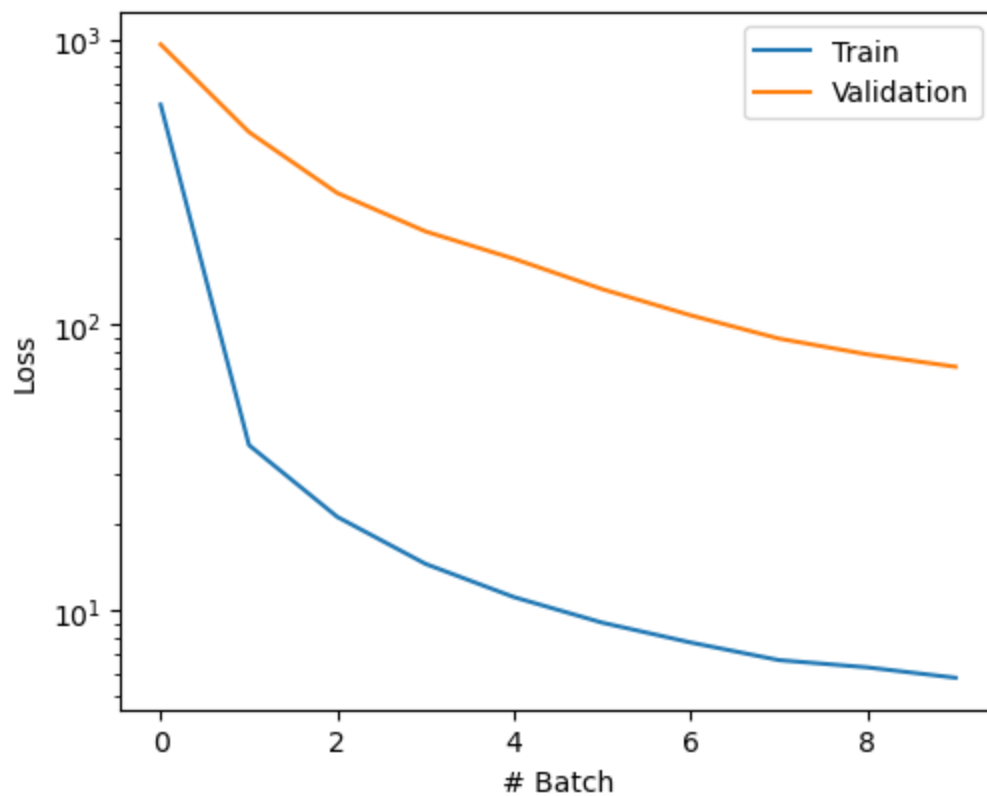
Validation loss: 72.41212649643421+/-1.198405645787716

Validation RMSE: 8.509531508633962+/-1.094717153326701

Test loss: 25.19259736686945

Test RMSE: 5.019222785140091





In [18]: ANI_KFold(model=AtomicNet, k=2, dataset=dataset, batch_size=256, learning_rate=1e-4

Fold 0:

Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 15/15 [15:02<00:00, 60.18s/it]

Fold 1:

Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 15/15 [15:30<00:00, 62.06s/it]

Final results:

Train loss: 3.1857178851337267+/-0.13953039751550023

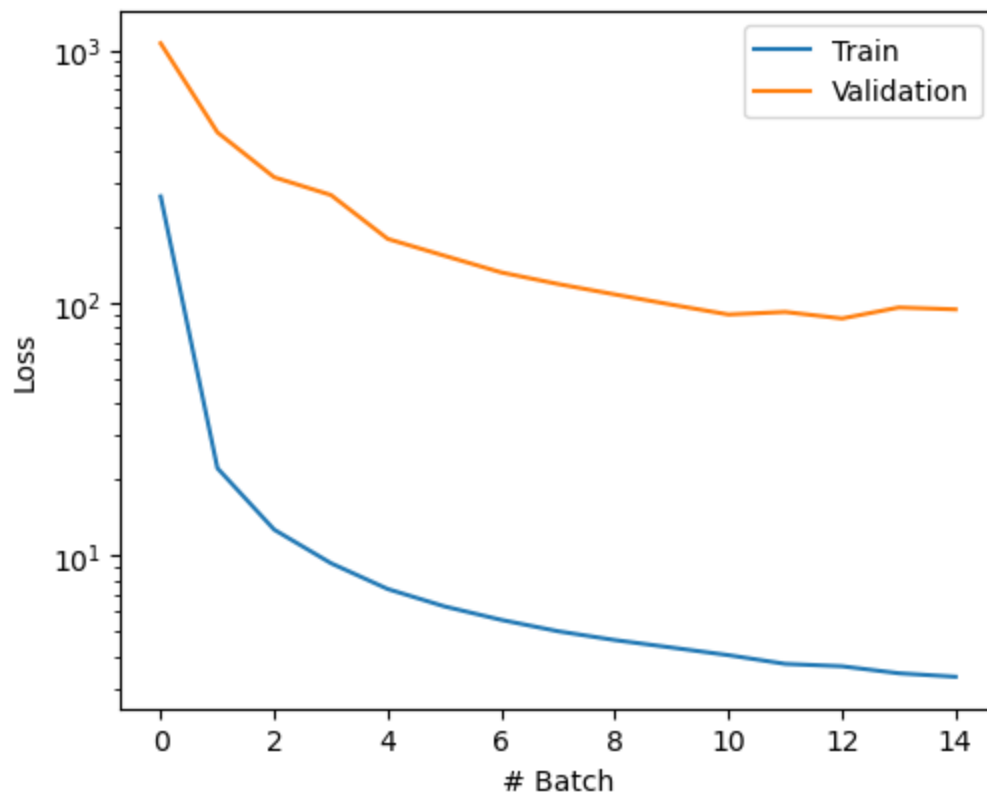
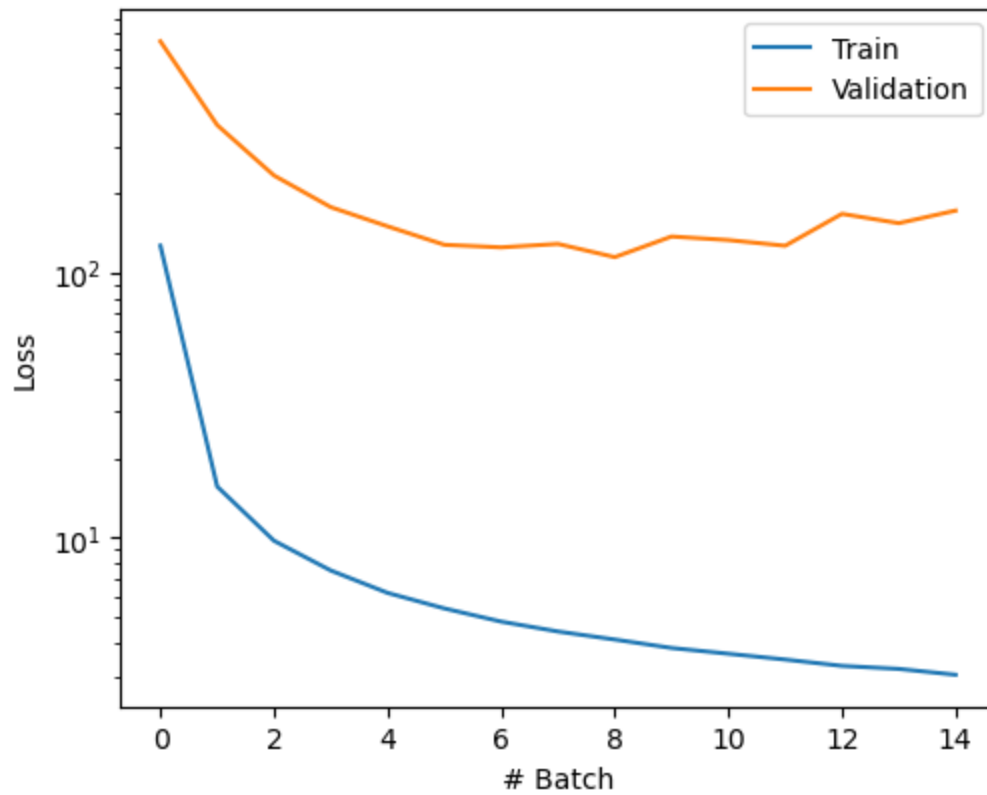
Train RMSE: 1.7848579453653242+/-0.37353767884311245

Validation loss: 100.6633315347135+/-13.722801949828863

Validation RMSE: 10.03311175731206+/-3.704430043856796

Test loss: 33.35384424030781

Test RMSE: 5.775278715378835



Results

Using the previously determined optimal settings ($k=5$, $\text{batch_size}=128$, $\text{learning_rate}=1\text{e-}4$, $\text{epoch}=50$, $\text{l2}=1\text{e-}6$), I trained the model on all the data. It took more than 5 hours though

and was cut off before the last fold, so I decreased the nuymber of folds, increased the batch size, and decreased the number of epochs for the model to run faster. The result was a faster run time but higher loss. I then adjusted these parameters to balance between run time and loss.

I used MSE as the loss function in my model, and RMSE is the square root of that, with the same units as the data, which is in kcal/mol.

My results were: training 1.78 kcal/mol, validation 10.03 kcal/mol, and testing 5.77 kcal/mol,

Compared to the paper's results of training 1.2 kcal/mol, validation 1.3 kcal/mol, and testing 1.3 kcal/mol.

My results had higher RMSE, and the validation and testing RMSE are still higher compared to the target of 5 kcal/mol. The condensed results are shown below.

Final results:

Train loss: 3.1857178851337267+/-0.13953039751550023

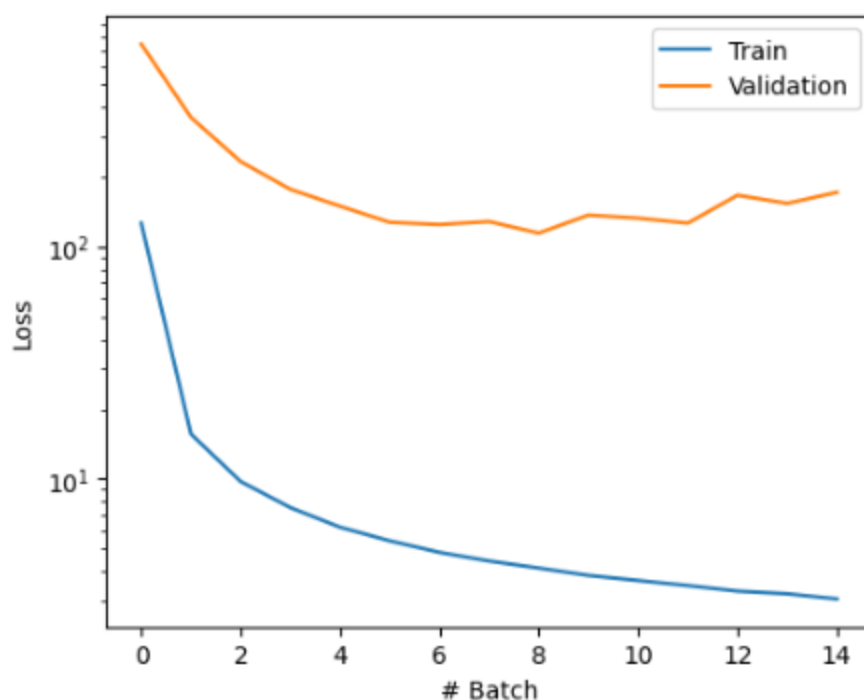
Train RMSE: 1.7848579453653242+/-0.37353767884311245

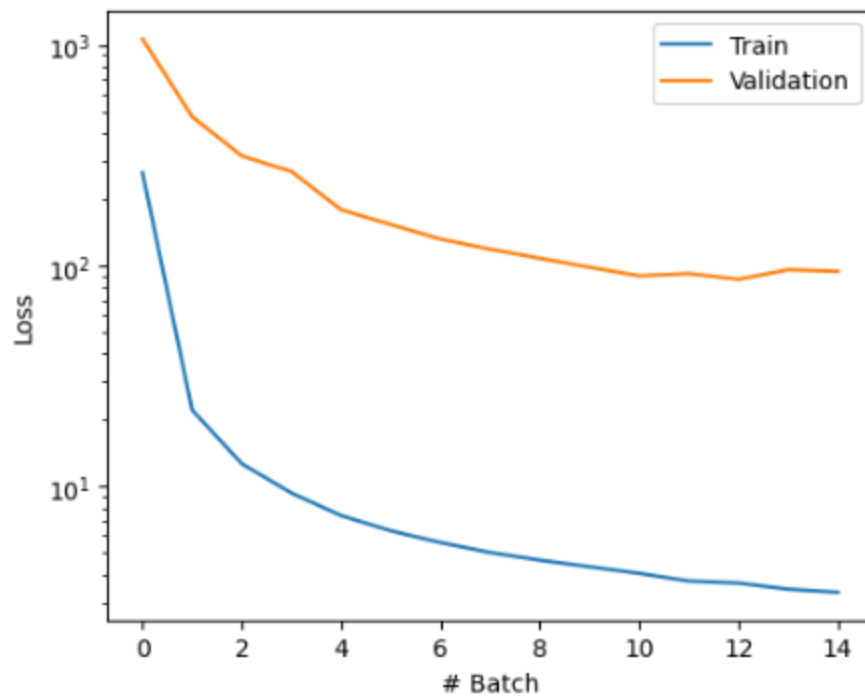
Validation loss: 100.6633315347135+/-13.722801949828863

Validation RMSE: 10.03311175731206+/-3.704430043856796

Test loss: 33.35384424030781

Test RMSE: 5.775278715378835





In []: